

Logica para Ciencia da Computacao

# Slitherlink em SAT

Codificacao CNF, eliminacao de subtours e comparacao CaDiCaL x Kissat

Arthur Oliveira

Joao Caique

Gustavo Alexandre dos Santos

# O problema

- Selecionar arestas de uma grade.
- Cada dica  $k$  exige exatamente  $k$  arestas ao redor da célula.
- Vertices possuem grau 0 ou 2.
- A solução deve ser **um único laço**.

## Desafio central

Dicas e graus são locais. Conectividade é uma propriedade global.

## Complexidade

Slitherlink é NP-completo; uma solução candidata pode ser verificada em tempo polinomial.

# Variaveis proposicionais

Uma variavel booleana para cada aresta horizontal e vertical:

$$x_e = 1 \Leftrightarrow e \text{ pertence ao laco}$$

$$N = (R+1)C + R(C+1) = 2RC + R + C$$

$$\text{idx}(h[i,j]) = 1 + iC + j$$

$$\text{idx}(v[i,j]) = 1 + (R+1)C + i(C+1) + j$$

# Clausulas de celula

## Exatamente 1

Uma clausula exige ao menos uma aresta.

Seis clausulas binarias proíbem qualquer par.

**Total: 7**

## Exatamente 2

Quatro triplas positivas exigem ao menos duas.

Quatro triplas negativas permitem no maximo duas.

**Total: 8**

Dicas 0 e 4 usam quatro unitarias; dica 3 e dual a dica 1.

# Graus e contagem

- Canto: 2 clausulas para permitir grau 0 ou 2.
- Borda: 4 clausulas para proibir graus 1 e 3.
- Interno: 9 clausulas para proibir graus 1, 3 e 4.
- Uma clausula global elimina a solucao sem arestas.

$$M_{base} = M_{grau} + 4n_0 + 7n_1 + 8n_2 + 7n_3 + 4n_4 + 1$$

# Por que ainda surgem varios lacos?

Grau 0 ou 2 garante que cada componente seja um ciclo, mas não conecta ciclos distintos.

## Contraexemplo

Um modelo SAT pode satisfazer todas as dicas e ainda conter dois ou mais subtours.

## Corte

$$\forall e \in S \neg x_e$$

Ao menos uma aresta do ciclo isolado deve mudar.

# Refinamento CEGAR

1. Resolver o CNF base.
2. Reconstruir o grafo das arestas positivas.
3. Detectar componentes por DFS.
4. Se houver um componente, aceitar.
5. Se houver varios, bloquear o menor subtour e repetir.

SAT → analisar → contraexemplo → corte → SAT → ...

O processo termina: cada corte elimina uma atribuicao e o conjunto de atribuicoes e finito.

# Implementacao e verificacao

## Modulos

- `gerador.py`: CNF e sanidade DIMACS
- `refinamento.py`: componentes e cortes
- `solver_runner.py`: solvers e metricas
- `visualizar.py`: antes/subtour/depois

## 14 testes

Parser, cardinalidade, cabecalho, literais, CNFs versionados, componentes e integracao real com CaDiCaL/Kissat.



# Resultados

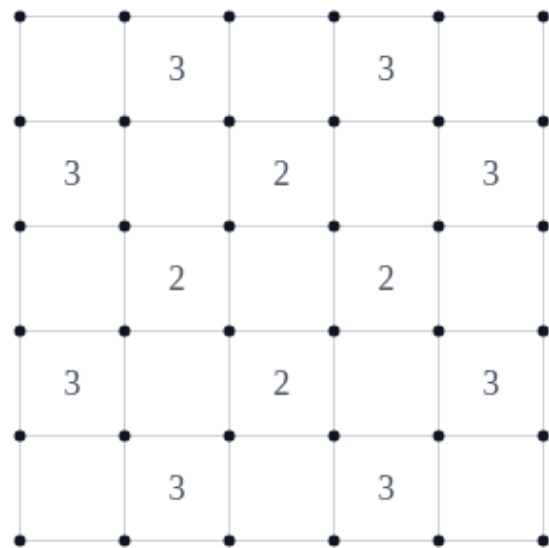
| Instancia   | N  | Base | CaDiCaL      | Kissat       |
|-------------|----|------|--------------|--------------|
| 3x3 SAT     | 24 | 99   | SAT, 1 iter. | SAT, 1 iter. |
| 4x4 UNSAT   | 40 | 186  | UNSAT        | UNSAT        |
| 5x5 SAT     | 60 | 305  | SAT, 2 iter. | SAT, 3 iter. |
| 6x6 UNSAT   | 84 | 537  | UNSAT        | UNSAT        |
| 3x3 UNSAT   | 24 | 85   | UNSAT        | UNSAT        |
| Problema 11 | 60 | 266  | UNSAT        | UNSAT        |

Todos os tempos ficaram abaixo de 4 ms no experimento registrado.

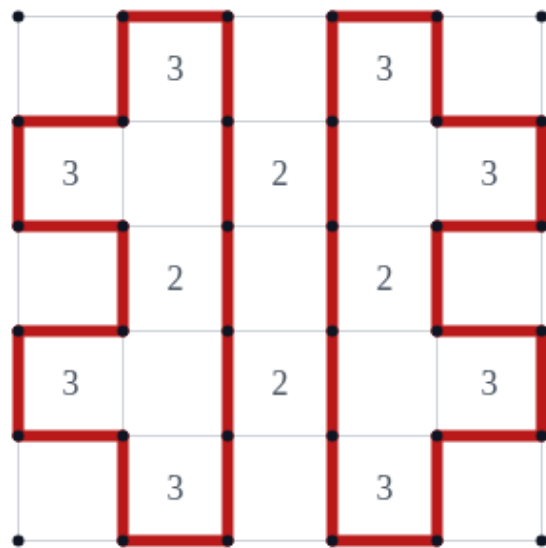
# Refinamento da instancia 5x5

## Slitherlink 5x5 - kissat

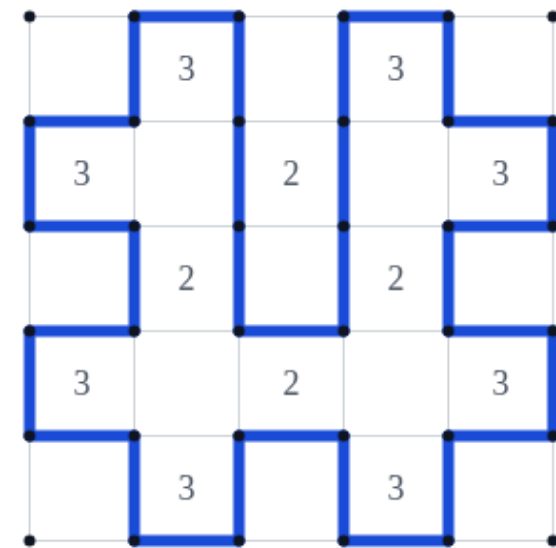
Instancia



Modelo com subtours



Laco final (3 iteracoes)



# Conclusões

- A codificação local é compacta e verificável.
- Conectividade global foi tratada sem inflar o CNF base.
- Solvers distintos exigiram quantidades diferentes de cortes.
- Uma formulação estática por fluxo ou alcance evitaria iterações, mas usaria mais variáveis.
- Código, resultados e relatório são reproduzíveis pelo repositório.

**Repositório:** [github.com/santos-ag/slitherlink-solver](https://github.com/santos-ag/slitherlink-solver)

# Perguntas?